



GSAS-II: the genesis of a modern open-source all purpose crystallography software package

Brian H. Toby and Robert B. Von Dreele*

Received 15 October 2012

Accepted 4 February 2013

Advanced Photon Source, Argonne National Laboratory, 9700 South Cass Avenue, Argonne, IL 60439-4814, USA. Correspondence e-mail: vondreele@anl.gov

The newly developed *GSAS-II* software is a general purpose package for data reduction, structure solution and structure refinement that can be used with both single-crystal and powder diffraction data from both neutron and X-ray sources, including laboratory and synchrotron sources, collected on both two- and one-dimensional detectors. It is intended that *GSAS-II* will eventually replace both the *GSAS* and the *EXPGUI* packages, as well as many other utilities. *GSAS-II* is open source and is written largely in object-oriented Python but offers speeds comparable to compiled code because of its reliance on the Python NumPy and SciPy packages for computation. It runs on all common computer platforms and offers highly integrated graphics, both for a user interface and for interpretation of parameters. The package can be applied to all stages of crystallographic analysis for constant-wavelength X-ray and neutron data. Plans for considerable additional development are discussed.

© 2013 International Union of Crystallography
Printed in Singapore – all rights reserved

1. Introduction

For the past 60 years crystallographers have depended on computer software to perform increasingly sophisticated crystallographic calculations. In the earlier days, it was common to use a program from one source to solve a crystal structure, one from a different source for refinement, another for distance–angle and error analysis and so on. Over time it has become more common to use crystallographic packages that contain programs for completion of multiple steps in the structure determination process. This made the life of the crystallographer easier, in that integration inherent in a package allowed information to flow smoothly through the work process and lessened the learning investment needed to master the individual steps.

Beginning in the 1980s, the *General Structure and Analysis System* (*GSAS*) crystallographic package was developed to address all aspects of crystallographic refinement and subsequent structural analysis for all types of X-ray and neutron diffraction data, from both constant-wavelength and time-of-flight instrumentation and including both powder and single-crystal studies, possibly in combination (Larson & Von Dreele, 2004). *GSAS* was the first comprehensive package of its type, and the only one that addresses needs for structures of all complexity, including proteins. However, *GSAS* did not address pre-refinement tasks, such as indexing powder data or structure solution. Newer packages, such as *FullProf*, *TOPAS* and *Jana*, have included a wider range of structure determination tools (Rodríguez-Carvajal, 1993; Petricek *et al.*, 2006; Coelho, 2007).

The authors of this article have long been involved with the *GSAS* package and its companion interface *EXPGUI* (Toby, 2001). The two codes are very widely used, with hundreds of new citations each year. However, it has become clear that it has been increasingly more difficult to advance their capabilities, since they are built with older programming languages and designs that are far less efficient for code development, code reuse and utilization of contemporary graphical capabilities than would be desired (Toby & Von Dreele, 2007). For these reasons, it was decided to create a new package, called *GSAS-II* and described herein, utilizing a new programming environment and

incorporating many aspects of the previous packages' strengths. This project was further motivated by the need for a single software package that encompasses every aspect of the crystallographic workflow, particularly for powder diffraction, starting with data reduction through to the domain of *GSAS* and similar packages. As *GSAS-II* has been developed, the advantages of code development in a graphics-rich environment have become clear. Not only is design of an easy to understand and efficient graphical user interface (GUI) a central development task, but the availability of excellent graphics tools allows visualization of results to also be a key initial design goal. Finally, it should be noted that the practice of crystallography has changed in the past few decades: the codes developed a half-century ago were written by expert crystallographers for themselves and their peers. Nowadays, crystallographic analyses are commonly performed as one part of a research investigation, often by scientists with considerably less education in crystallography. New tools must thus adapt, and as such *GSAS-II* is designed to lower the barriers experienced by new users.

2. GSAS-II programming environment

Modern software is written in a variety of programming languages, including object-oriented languages, such as C++ and Fortran2003, which are compiled and linked into processor- and operating-system-dependent executable images; another choice is a virtual-machine (VM) language, most commonly Java, which is compiled into a machine-independent form; the third choice is scripting languages, such as JavaScript, Perl or Python, which are interpreted at the time of execution. Each choice has advantages, in that codes written in compiled languages usually run fastest, while virtual-machine codes tend to be most portable and easy to distribute but will be less computationally efficient. Scripting languages usually allow for the most rapid code development. Since it is interpreted at the time of execution rather than compiled, usually the same code will run on a variety of platforms but may be computationally inefficient.

However, as will be discussed, with care it is possible to create Python script code that runs as fast as carefully written compiled code where speed matters.

Distribution of both VM and script-based programs requires installation of additional software that runs the VM or script interpreter. In the case of a scripting language, this interpreter can be a fairly massive set of software. However, this interpreter allows for code to be revised locally, as it is interpreted when the program is started. An example of how this can introduce new capabilities is described below when discussing *GSAS-II* data import support.

The Python programming language was selected for *GSAS-II*, as it is one of the most modern and it has a wealth of available packages that further augment the native features of the language. Furthermore, we have found that code written in Python is quite portable between Windows, Mac OS X and Linux platforms. In the few rare cases where sections of *GSAS-II* are found to run differently between platforms, the cause is usually a subtle programming error that, when corrected, addresses the incompatibility. Efficiency of code execution in Python has not posed much of a challenge either. For the vast majority of the code in *GSAS-II*, speed of execution is of little import. It does not matter if a screen is repainted in 10 or 100 μ s after a button is pressed, since a user will take seconds to see the change and respond. As will be discussed later, algebraic computations can be very quick in Python, but in the few cases where we were unable to perform computations sufficiently quickly in Python, we have written short Python-wrapped Fortran subroutines that are compiled and are called from Python modules. This was also done in the case of the *NRCVAX/GSAS* group symmetry analysis code written by Allen Larson for space group operator derivation, as it was found to be very difficult to port to Python.

Python contains a large number of standard packages, but in addition there are a very large number of add-on packages, which are distributed by external groups, that further augment the power of the computational environment. A small number of these packages are used in the *GSAS-II* code:

(1) NumPy and SciPy (<http://www.numpy.org>). While the origins of these packages are separate, their development is now integrated.

Together they allow Python to offer a sophisticated data structure for numerical processing and many valuable routines for computation. Most routines operate at speeds comparable to those of highly optimized C, C++ or Fortran code, which allows most *GSAS-II* computations to run very efficiently.

(2) wxPython (<http://www.wxpython.org>) is a Python interface to the platform-independent *wxWidgets* GUI package. It offers a wealth of graphical primitives that are implemented with the native 'look-and-feel' of the underlying platform. Thus *GSAS-II* will behave like a Windows application on that platform, but will behave more like a Mac application on that platform.

(3) matplotlib (<http://www.matplotlib.org>) is a two/three-dimensional plotting library for scientific graphics. It allows for quite sophisticated on-screen and publication-quality graphics. *GSAS-II* plots are 'zoomable' for examination of details and can be exported in common graphical file formats.

(4) PyOpenGL (<http://pyopengl.sourceforge.net>) is a binding of the three-dimensional OpenGL graphics language. It is used in *GSAS-II* for rotatable three-dimensional display of crystal structures and superimposed Fourier maps.

These packages were selected on the basis of the maturity and power offered by each. The requirement for inclusion of these additional packages produces a disadvantage in code distribution, as one must install both Python and these packages before *GSAS-II* can be used. However, Python and all but one of the selected packages can be installed *via* a single download for all common computers using the 'EPD Free' Python package that Enthought Inc. distributes at no cost (http://www.enthought.com/products/epd_free.php). PyOpenGL is not included in EPD Free, but the distribution files for *GSAS-II* include the installation package for PyOpenGL. When *GSAS-II* is first started, if PyOpenGL is not present, it is then installed automatically.

It should be noted that there are currently two generations of the Python language, versions 2.x and 3.x, which differ slightly. *GSAS-II* uses the 2.x generation syntax, since not all of the required packages have yet been adapted for 3.x. When all packages required by *GSAS-II* have been shown to work reliably in Python 3.x, and suitable distributions are available, then it is expected that an update to 3.x can be accomplished quickly.

3. *GSAS-II* GUI design

GSAS-II uses four distinct windows for operation (Fig. 1). The main window shows the contents of the current project or workspace in hierarchical (tree or outline) format. At the base level of the tree, there are a number of entries for overall project settings, plus a base-level entry for structural phase(s), and another base-level entry is added for each data set as it is entered. Each phase is represented by a subentry under the Phases base-level entry. Note that there is no limit to the number of phases or data sets that can be included in a project and any phase or data set may be deleted from the project. For each data set's base-level entry in the data tree, there are a number of subentries for background parameters, instrumental parameters, sample parameters *etc.* Associated with the data

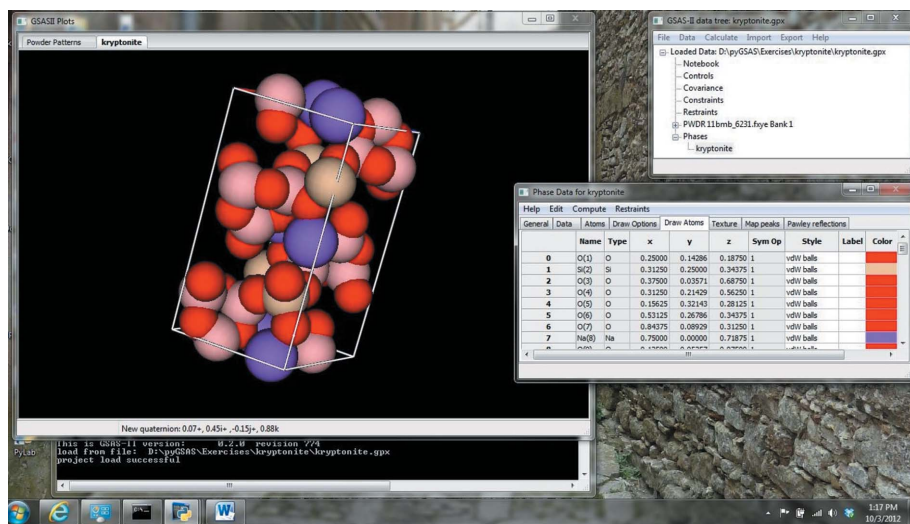


Figure 1

A screen shot of the four *GSAS-II* windows. The *GSAS-II* data tree window is in the upper right, the data window is below it, the plot window showing the crystal structure corresponding to the atom coordinates shown in the data window is to the left, and the *GSAS-II* console window is largely obscured behind the plot window.

tree window is a menu bar with entries for global operations on the entire project (e.g. input/output of data, phases and refinement).

The second window is for display and editing of parameters. The content of this window is determined by the item selected in the data tree window. On Windows and Linux, this window also has a menu bar with menu items specific to the type of data contained in the window, but on a Macintosh, where the system design allows only one menu bar to be displayed, all menus are combined into a single menu that is displayed in the customary way for that platform.

The third window is used for display of two- and three-dimensional graphics. Many of the project tree items have an associated graphics display; each is shown in its own tab on the plot window and one can switch tabs at will to view any of the generated plots. Thus, if a powder data set is selected, the pattern (or all patterns) is plotted here. Likewise, if the entry for parameter covariance is selected, the covariance matrix is displayed graphically in this window but in a separate tab from the powder one. Selecting a data set's instrumental parameters creates a plot of peak widths (as $\Delta d/d$) against 2θ or Q , etc. Different options for a phase's atomic parameters allow plotting of the structure in a variety of formats. Selection of texture parameters causes plotting of a pole figure or its inverse. In all cases the plot may be saved in a format (e.g. BMP, TIFF, JPEG etc.) appropriate for inclusion in other documents (e.g. pasted into a Microsoft Word document).

The fourth window is a console window. This shows status and error messages from *GSAS-II*. It is used only for output. All user input is provided in the data window or in pop-up dialog boxes; the user never types any input into the console. However, one can, by using operating-system-specific commands, select and copy text from this window and paste it elsewhere (e.g. an open text editor window).

4. *GSAS-II* capabilities

The capabilities of *GSAS-II* as of this writing are described in the following paragraphs. A later section describes some anticipated future additions.

4.1. Area detector data reduction

Powder diffraction data are being collected increasingly with area detectors. Area detector data sets can be collected in a fraction of a second, improve crystallite sampling statistics with grainy samples and reduce (or help quantify) preferred orientation. When data are collected with an area detector, one must calibrate the positioning of the detector so that every pixel on the detector can be mapped to a known 2θ and azimuthal angle. Measurement of the diffraction pattern from a standard with known peak positions allows calibration. From this measurement, the distance from the sample to the detector, the relative position of the beam center, and the tilt angles between the X-ray beam and the detector normal are determined. Once the calibration constants are known, data can be integrated around the Bragg rings, using either ranges of azimuthal angles or all observed pixels subject to optional masking to remove unwanted features. Software for this process has been available for many years (e.g. *Fit2D*; Hammersley, 1998). *GSAS-II* also performs these operations but improves upon this process by treating the projection of a Bragg cone rigorously, even when the detector is considerably tilted from the normal to the X-ray beam. *GSAS-II* also simplifies the calibration and integration process, even when the incident beam is not projected onto the face of the area detector. Furthermore, the integration result is immediately available within *GSAS-II* for further processing (e.g. peak picking/fitting, indexing and Rietveld refinement).

A calibration data set may be read in along with any number of experimental data sets. Calibration is completed by indicating a choice for the standard, specifying the X-ray wavelength, and then selecting five or more points on the first diffraction ring with the mouse. A calibration refinement is then completed in a few seconds. The resulting calibration constants can be copied to all the experimental data sets in the project in a single step or can be saved for use in a different project. Once calibration is complete, all images read into a project can be integrated in a single step.

Most area detector data formats that have been encountered from devices and software at the APS have used variants of the TIFF format, many with incomplete or incorrect headers. Input routines have been constructed to recognize and read these formats, filling in the missing or incorrect data. In addition, *GSAS-II* will read MAR, ADSC and GE format files. Files can be read from inside zip archives.

4.2. Data input

Import of structural parameters, one-dimensional powder diffraction data and single-crystal data sets is performed *via* a generalized object-oriented layer that is self-configuring on the basis of the data import routine files that are found when *GSAS-II* is started. These files are identified by their names. This allows for support of additional formats through inclusion of new Python routines, without any other changes to the code. A data import routine may be as short as 30 lines of code. Experience with a similar capability in *EXPGUI* and other code has shown that users with modest programming skills are well able to add support *via* new input routines for previously unsupported file formats by using the supplied import routines as examples. The base data import class provides the ability to extract and read data files that are inside zip archives, so this capability is automatically provided for any new formats that may be added.

GSAS-II currently supports the same input file formats as *GSAS* for powder diffraction and single-crystal data and can import structural data from *GSAS* experiment (.EXP) files. *TOPAS* powder data files are also supported. In addition, structures and diffraction data can be imported from a CIF, through use of the *pyCIFRW* library (Hester, 2006), which is redistributed with *GSAS-II*.

4.3. Peak fitting

GSAS-II allows fitting of the diffraction peaks in a selected region of a pattern, where peak positions are not constrained by lattice constants and can be refined independently. Use is made of the familiar description of an axial-divergence-broadened (Finger *et al.*, 1994) pseudo-Voigt (Thompson *et al.*, 1987) powder peak profile function; this is implemented *via* a modification of the Fortran code used in *GSAS*. The Gaussian and Lorentzian widths for each fitted peak may be refined individually or can be constrained by profile equations with refined parameters (e.g. U , V , W , X , Y). The latter mode is usually preferred because it results in better peak positions for highly overlapped peaks for which refinement of individual profiles would be difficult.

4.4. Indexing

Powder pattern indexing is available using a minor variation of the method of Coelho (2003); it can treat any Laue symmetry and is not bound by unit-cell size.

4.5. Intensity extraction

When the unit cell(s) for a powder refinement are assigned, it is possible to perform a Pawley (1981) intensity extraction. This is similar to a Rietveld fit, except that reflection structure factors are treated as refined variables. Highly overlapped reflections are

combined *via* automatic constraints, partitioning the intensity equally amongst the contributors; restraints to prevent negative intensities may optionally be used.

4.6. Structure solution

The first structure solution tool to be implemented in *GSAS-II* is the method of charge flipping (Oszlányi & Sütő, 2004). This can be used with either single-crystal structure factors or those extracted from powder data *via* a *GSAS-II* Pawley refinement or preliminary Rietveld refinement based, for example, on a partial model. As a demonstration of the power of the NumPy package for efficient coding and computation, we offer in Appendix A an annotated code snippet that constitute the entire charge-flipping algorithm, as implemented in *GSAS-II*. A preliminary step that is not shown is to expand the array of reflections to cover the entire sphere of reciprocal space and zero fill to the limit defined by the desired map resolution (typically 0.5 Å); a random distribution of phases are assigned to the reflections.

In the charge-flipping algorithm, the user has the option to normalize structure factors against a selected elemental form factor; their sum is then computed. The user also selects two charge-flip controls: one is the conventional lower level of the density below which the density is flipped in sign, and the other is an upper level above which the density is flipped in sign. The latter control provides a means of suppressing those spurious charge-flip solutions that consist of a single very strong density peak (a so-called 'uranium atom solution'). The entire calculation is performed in double precision. This code snippet operates in a Python *while* loop. It is cycled repeatedly until no further improvement is seen by the user. It should be noted that, while execution of each Python statement adds some processing overhead that might not be encountered in a compiled language, each computation is performed *via* a call to the highly optimized NumPy module. Thus, since only a small number of statements are looped in each charge-flip cycle, the code runs at a speed comparable to a Fortran or C++ implementation. The brief length of this code shows that it is also relatively easy to write, in comparison to other languages. The computations are completed at a rate of ~1 s per charge-flip cycle on a reasonably new Intel i5 quadprocessor laptop running Microsoft Windows 7 with $\sim 5 \times 10^5$ structure factors/density points. Typically a solution is found within a minute (<100 cycles).

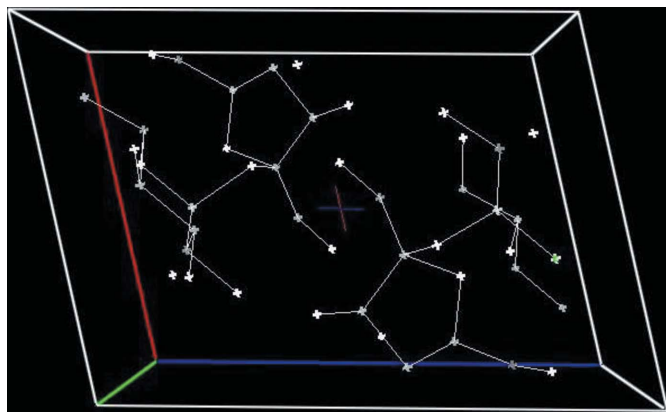


Figure 2

A charge-flip structure solution for sucrose from powder diffraction data collected at 30 keV (0.41326 Å) on APS beamline 11-BM. Diffraction peak intensities to $d_{\min} = 0.978$ Å were obtained *via* a Pawley refinement after peak picking and indexing by *GSAS-II*.

The resulting charge-flip density map has an arbitrarily placed origin with respect to the symmetry elements for the space group. The *GSAS-II* code makes use of the phase angle relationships in the charge-flip reflection set to determine the origin. Fig. 2 shows the resulting atomic positions obtained from a charge-flip solution for sucrose from high-resolution powder diffraction data collected at the APS 11BM beamline.

4.7. Fourier maps

Fourier computations *via* NumPy fast Fourier transform (FFT) methods may be carried out with observed or computed structure factors or their differences using the phases from the computed structure factors. Patterson maps may also be computed. The Fourier maps can be visualized by superimposing them on the atom structure for structure completion and final verification. Rather than computing contours, the map is displayed using points with radius proportional to the Fourier density and with a color determined by the density sign. Users select the cutoff for the density representation *via* a slider in the appropriate GUI, and users can indicate that locations of map features should be added to the atom array as positions are assigned.

4.8. Peak-shape model

GSAS-II employs the same pseudo-Voigt peak shape used for single peak fits noted above. This is similar to that used in *GSAS*; however, the Finger–Cox–Jephcoat (Finger *et al.*, 1994) asymmetry correction is simplified by assuming that the sample height (2S) is the same as the detector height (2H) and thus only a single parameter is needed. Furthermore, in the *GSAS-II* formulation, there is a complete decoupling of instrument broadening parameters from those of the sample, even though this does lead to some redundancy. However, if the instrumental parameters are determined initially using an appropriate standard, then the refined sample terms directly represent physically meaningful parameters, such as sample displacement, strain and crystallite size.

GSAS-II allows for refinement of two sample displacement parameters in Debye–Scherer geometry, parallel and perpendicular to the beam. Strain and crystallite broadening can each be expressed in terms of anisotropic coefficients and three-dimensional surfaces can be displayed to interpret these coefficients.

4.9. PDF computation

When data are collected over a sufficiently wide *Q* range, it is possible to compute in *GSAS-II* a pair distribution function (PDF) from a set of X-ray diffraction data. This computation uses NumPy FFT methods and has been simplified by combining several common corrections into a single function with a single *ad hoc* correction term, since many of these corrections have approximately the same functional form. The value for this correction term can be varied *via* a slider to easily minimize spurious peaks at short distances and provide a visual sensitivity analysis to confirm that PDF features are not influenced by these corrections. It is anticipated that there will be additional improvements for this module.

4.10. Refinement modes

There is no limit to the number of data sets or phases that can be included in a single *GSAS-II* project. When a refinement will be performed using a large project, it is possible to perform a single refinement, where all parameters are simultaneously best fit to all data sets. However, it is also possible to perform a sequential fit, where the results from the fit to the first data set are used as a starting point for the second data set, and so on.

4.11. Constraints

GSAS-II offers constrained refinements. Constraints reduce the number of refined parameters by creating formal relationships between parameters in the model. Most commonly, constraints are applied to atomic coordinates, occupancies or displacement parameters, some of which are generated automatically from symmetry considerations, but others may be added by the user to force the model to comply with assumptions posed as part of the analysis (for example, that a site is fully occupied, or that chemically similar atomic environments are well represented by only a single value for U_{iso}). All *GSAS-II* parameters may be used in constraints and, unlike *GSAS*, a parameter may be used in more than one constraint. Constraints may be applied in several ways; parameters may be equated,

$$p_1 = c_2 p_2 = c_3 p_3 = \dots, \quad (1)$$

or new composite variables can be created from a series of parameters,

$$v_0 = c_1 p_1 + c_2 p_2 + c_3 p_3 + \dots, \quad (2)$$

or one or more linear equations can be input to reduce the number of degrees of freedom in a model,

$$c_0 = c_1 p_1 + c_2 p_2 + c_3 p_3 + \dots. \quad (3)$$

In these equations, c_i is a constant, p_i is a parameter in the model and v_i is a new variable that is created for the refinement. Note that, in the latter two cases, the program creates a full set of n linear equations for each of the n parameters contained in a set of constraints. It is the variables created by these equations that will be varied, depending on the settings for the parameters used in these equations.

4.12. Restraints

Restraints can be introduced in *GSAS-II* on bond lengths, angles, planes, chiral group volumes, torsion angles and paired torsion angles. The latter two are implemented as a restraint on a pseudo-potential based on allowed values of bond torsion angles found in the literature as described by Von Dreele (1999, 2005). Restraints on the sum of the unit-cell contents (e.g. chemical composition) can also be formed. Restraints are then implemented as additional observations in the formation of the least-squares Hessian matrix as described below.

4.13. Quantitative phase analysis

GSAS-II can readily be used for quantitative phase analysis via the Rietveld method for mixtures of any number of phases. Each phase in a powder pattern is assigned a scale factor based on the unit-cell contents. Conversion to weight fractions, with assigned standard uncertainties, is provided as a result of the Rietveld refinement.

4.14. Refinement minimizers

Refinement is performed using one of three minimizers. Most users will use the default, which is a Levenberg–Marquardt least-squares minimizer that has been developed for *GSAS-II*. For this minimizer, analytic or numerical derivatives are computed for all parameters in the model and an $n \times n$ Hessian matrix is computed, where n is the number of refined parameters. The Hessian is then normalized by the diagonal elements to avoid the effects of the extreme range of values frequently encountered in a Rietveld refinement; this normalization is retained for recovering the inverse and computing the solution. In the Levenberg–Marquardt algorithm the diagonal of the Hessian matrix is modified to reduce the impact of correlation iteratively when a fit is found to diverge. In the limit where this modification is taken to the extreme, the refinement would occur by steepest-descent

minimization, while in the case where correlation is negligible, a Gauss–Newton minimization results. The latter is better and faster at finding minima but fails for highly correlated parameters (Prince, 2004). When there is complete correlation between variables, or a variable is found to have no impact on the model, this singularity is addressed by automatically removing variable(s) from the refinement. For this minimizer, the model computation (e.g. structure factors and powder profiles) is separate from the derivative computation; this is in contrast to the methods used in older refinement code (e.g. *GSAS*) where the function and its derivatives are computed simultaneously. In practice, we have found this implementation of the Levenberg–Marquardt minimizer succeeds on all refinements we have attempted, even when correlation is extremely high.

The SciPy leastsq minimizer, using either supplied analytical derivatives or internally computed numerical derivatives, is also available. In that case an $n \times m$ Jacobian (or design) matrix is constructed, where n is the number of refined parameters and m is the number of observations. For larger multiple data set problems, m can be greater than 10^6 , which requires more memory than found in most contemporary computers. This NumPy minimizer is based on the well trusted MINPACK (<http://www.netlib.org/minpack>) `lmdr` and `lmdif` routines, which perform well, although the numerical derivative version (`lmdif`) is much slower than the analytical derivative version (`lmdr`).

5. Comparison with GSAS

The *GSAS* program is used, where possible, to validate code developed for the new capabilities in *GSAS-II*. Where models can be comparably parameterized (for Rietveld and single-crystal structure refinements), *GSAS* and *GSAS-II* give essentially identical results within the limits given by the standard uncertainties for identical models. Minor differences in detail arise from the use of single precision in Fortran77 throughout *GSAS* and double precision as provided by Python.

6. Future development

At present *GSAS-II* comprises ~35 000 lines of Python code; we envision considerable expansion to introduce new features. Export of results via CIF and other style formats is anticipated for early development as they are essential for user publication of results from *GSAS-II*. Fitting to neutron time-of-flight data is currently planned, as well as inclusion of neutron magnetic scattering and support for modulated structures. Monte Carlo simulated annealing of molecular models as an alternative structure solution technique and refinement of rigid body descriptions for these molecules are currently under development. The constraints module may be expanded to allow generation of parameters from fitting equations for parametric refinements or, better, parametric results may be fitted in a secondary step to the fitted variables and their covariances.

It is our hope that, since *GSAS-II* is written in an open fashion, the code can be used for more sophisticated structural modeling capabilities than are available at present, as may be needed to treat materials with deviations from long-range ordering, or may be used to fit other types of data where quantitative simulation is possible from atomistic data, for example with other types of physical measurements. We also hope to explore fitting approaches to diffraction data where continuum models, as opposed to atomistic ones, are used. The most obvious cases for this are reflectometry and small-angle scattering. User modifications are welcome for possible

inclusion in the distributed version of *GSAS-II* through submission of code, but write access to the code repository can be granted to external collaborators upon request.

7. Distribution and documentation

At present, *GSAS-II* is being enhanced aggressively and we expect this to continue for a period of years; changes are made on a sometimes daily basis, so it is best for users to run the latest version of the software. For this reason we prefer that users install the software directly using the Subversion protocol from the server at Argonne National Laboratory used for our development (<https://subversion.xray.aps.anl.gov/pyGSAS>), so that they may update to the latest version easily and often. When the Subversion program (`svn` or `svn.exe`) is in the program path, a help menu item allows *GSAS-II* to self-update. However, the need to install Subversion and Python as prerequisites for installing *GSAS-II* increases the complexity of software installation. This has been somewhat streamlined through provision of a download script (Mac and Linux) or an installation kit for Windows, but multiple downloads are still required. When *GSAS-II* development is more mature, we hope to further streamline *GSAS-II* installation.

Documentation for *GSAS-II* is provided through contextual-based help entries, tied to each type of *GSAS-II* window, and *via* worked-through tutorial examples. This documentation is included in the software distribution but can also be accessed *via* the *GSAS-II* home page (<https://subversion.xray.aps.anl.gov/trac/pyGSAS>). Installation instructions are provided as wiki pages referenced on the *GSAS-II* home page.

Users are also encouraged to subscribe to a mailing list (see <http://www.aps.anl.gov/mailman/listinfo/GSAS-II>) where announcements about software updates are posted and where users can post messages to each other.

APPENDIX A

Example of Python/NumPy code

The following annotated example section of Python/NumPy code implements the iterated part of the charge-flipping algorithm in *GSAS-II*.

```
1CErho=np.real(fft.fftn(fft.fftshift(CEhkl)))*(1.+0j)
2CEsig=np.std(CErho)
3CFrho=np.where(np.real(CErho)>=flipData['k-factor']*CEsig,CErho,-CErho)
4CFrho=np.where(np.real(CErho)<=flipData['k-Max']*CEsig,CFrho,-CFrho)
5CFhkl=fft.ifftshift(fft.ifftn(CFrho))
6CFhkl=np.where(CFhkl,CFhkl,1.0)
7phase=CFhkl/np.absolute(CFhkl)
8CEhkl=np.absolute(Ehkl)*phase
9Ncyc+=1
10sumCF=np.sum(np.ma.array(np.absolute(CFhkl),mask=Emask))
    DEhkl=np.absolute(np.absolute(Ehkl)/sumE-np.absolute(CFhkl)/sumCF)
    Rcf=min(100.,np.sum(np.ma.array(DEhkl,mask=Emask)*100.))
```

Notes: (1) This computes the real density; the shift is to accommodate the fact that the structure factor array is offset so that the indices are all zero or positive. `CEhkl` is a copy of the original structure factors `Ehkl` zero filled to the resolution limit. `np` refers to the NumPy package and `fft` to the fast Fourier transform package within NumPy. (2) This computes the standard deviation of the density map. (3) This performs the charge flip when the density is less than the level defined by the user. (4) This performs the charge flip when the density is larger than a level defined by the user and thus avoids the 'uranium atom' problem frequently encountered in charge-flipping solutions. (5) This computes structure factors based on the modified density. The shift is used to offset the results. (6) This is used to avoid a potential zero divide error in the next step. (7) This extracts the phase angle from the computed structure factors. (8) This applies the phases to the original expanded structure factors. (9) This counts charge-flip cycles. (10) These last three steps compute the charge-flip residual using only the observed reflection set; the mask removes those added to zero fill the array and `sumE` is the sum of the original structure factors.

Use of the Advanced Photon Source, an Office of Science user facility operated for the US Department of Energy (DOE) Office of Science by Argonne National Laboratory, was supported by the US DOE under contract No. DE-AC02-06CH11357. We would like to thank James Hester for his PyCIFRW package, which greatly simplified integration of CIF support, and we thank the APS for its continuous support of the *GSAS-II* Subversion server and web site/wiki pages.

References

- Coelho, A. A. (2003). *J. Appl. Cryst.* **36**, 86–95.
- Coelho, A. (2007). *TOPAS*. Bruker AXS GmbH, Karlsruhe, Germany.
- Finger, L. W., Cox, D. E. & Jephcoat, A. P. (1994). *J. Appl. Cryst.* **27**, 892–900.
- Hammersley, A. P. (1998). *Fit2DV10.3 Reference Manual V4.0*. Internal Report ESRF98HA01T. ESRF, Grenoble, France.
- Hester, J. R. (2006). *J. Appl. Cryst.* **39**, 621–625.
- Larson, A. C. & Von Dreele, R. B. (2004). *General Structure Analysis System (GSAS)*. Los Alamos National Laboratory, New Mexico, USA.
- Oszlányi, G. & Sütő, A. (2004). *Acta Cryst.* **A60**, 134–141.
- Pawley, G. S. (1981). *J. Appl. Cryst.* **14**, 357–361.
- Petricek, V., Dusek, M. & Palatinus, L. (2006). *Jana2006*. Institute of Physics, Prague, Czech Republic.
- Prince, E. (2004). *Mathematical Techniques in Crystallography and Materials Science*. New York: Springer-Verlag.
- Rodríguez-Carvajal, J. (1993). *Physica B*, **192**, 55–69.
- Thompson, P., Cox, D. E. & Hastings, J. B. (1987). *J. Appl. Cryst.* **20**, 79–83.
- Toby, B. H. (2001). *J. Appl. Cryst.* **34**, 210–213.
- Toby, B. H. & Von Dreele, R. B. (2007). *IUCr Commission on Crystallographic Computing Newsletter*, No. 8, pp. 57–73.
- Von Dreele, R. B. (1999). *J. Appl. Cryst.* **32**, 1084–1089.
- Von Dreele, R. B. (2005). *Acta Cryst.* **D61**, 22–32.